Project Report

Dockerized Sports Rust Service

Prepared according to the Medicaps University – Datagami Skill Based Course report format

Course Name	DevOps
Institution Name	Medicaps University – Datagami Skill Based Course
Group Name	Placed BtechGroup07
Project Number	DO-50
Academic Year	2025-2026

Student Name(s) & Enrolment Number(s)

Sr. No.	Student Name	Enrolment Number
1	Chetan Oswal	EN22CS301295
2	Irya Patni	EN22CS301436
3	Uday Dubey	EN22EL301058
4	Himani Jaiswal	EN22CS301423

Report Overview

The Dockerized Sports Rust Service is an academic DevOps project that demonstrates how a modern backend application can be designed, containerized, and executed in a consistent and production-oriented manner. The project combines a static frontend served through Nginx, a Rust-based API built using Axum and Tokio, and a PostgreSQL database for persistence. These parts are orchestrated through Docker Compose so that the overall system starts in a controlled order and behaves consistently across development environments.

The main academic value of the project lies in showing how software engineering and DevOps principles come together in a practical implementation. Instead of focusing only on application logic, the project pays attention to build reproducibility, runtime isolation, service dependencies, environment handling, deployment readiness, and maintainability. This makes the project a good representation of full lifecycle thinking rather than only coding a backend service.

This report follows the structure expected in the provided report format while also expanding the narrative into a fuller project document. It covers the problem context, project objectives, scope, proposed solution, architecture, module-level design, technology choices, API design, database considerations, DevOps pipeline, operational benefits, outcomes, conclusion, and future scope.

Problem Statement & Objectives

1. Problem Statement

Traditional local application development often suffers from one recurring challenge: the application may run on one machine but fail on another because of differences in dependencies, runtime libraries, environment variables, or database setup. In backend systems, this problem becomes even more visible when multiple services are involved. A developer may need a web server, an application runtime, and a database, each with specific versions and configuration. If these are not standardized, deployment becomes error-prone and debugging takes unnecessary time.

The Dockerized Sports Rust Service addresses this problem by packaging the complete sports management backend into a containerized architecture. The project creates a consistent execution model where the frontend, API, and database are separated into clear layers but can still communicate in a predictable way over an isolated Docker network. The result is a system that is easier to run, easier to maintain, and better suited for DevOps-style automation.

From an academic perspective, the problem is not only about managing sports data such as teams and related records. It is also about demonstrating how a real application should be structured so that it can move smoothly from development to testing and toward deployment without major rework.

2. Project Objectives

- To design and implement a backend service in Rust for managing sports-related data through REST APIs.
- To containerize the application using Docker so that the runtime environment remains consistent across systems.
- To use Docker Compose for orchestrating the frontend, API, and PostgreSQL database as an integrated multi-container solution.
- To apply layered design principles that separate request handling, business logic, and persistence responsibilities.
- To demonstrate secure and efficient reverse proxy communication using Nginx in front of the backend API.
- To introduce DevOps-oriented practices such as health checks, multi-stage builds, CI pipeline steps, and deployment readiness.
- To prepare the project for future scalability using Kubernetes deployment and service manifests.

3. Scope of the Project

The project scope includes the design and implementation of a containerized sports service with a working frontend, backend, and database. The service supports creation and retrieval of team data, JSON-based communication, and persistent storage using PostgreSQL. The project also includes Docker and Docker Compose files required to build and run the application, along with initial Kubernetes configuration and a continuous integration workflow.

The present scope does not cover advanced authentication, role-based authorization, rich sports analytics, distributed logging, or a full enterprise deployment pipeline. The project is best understood as a clean and practical DevOps-centered foundation that can later be extended into a larger production-grade sports platform.

Proposed Solution

1. Project Summary

The proposed solution is a three-tier application in which presentation, application logic, and data persistence are separated into independent components. A browser accesses the application through an Nginx frontend running on port 3000. API traffic is proxied from the frontend container to the Rust backend service running on port 8080. The backend then communicates with the PostgreSQL database on port 5432 for persistent storage of sports data.

This architecture improves modularity because each tier has a focused responsibility. Nginx serves static files and handles reverse proxy routing, the Rust service manages API processing and validation, and PostgreSQL provides structured persistence. Docker Compose creates a private service network and coordinates startup order, while health checks ensure that the backend waits for the database to become available.

2. Key Features

- Three-tier architecture with clear separation of frontend, backend, and database.
- Rust backend using Axum, Tokio, Serde, and SQLx for API handling and asynchronous database interaction.
- PostgreSQL as a persistent relational database for storing structured sports data.
- Nginx reverse proxy for serving static frontend assets and forwarding API requests internally.
- Docker multi-stage build to create a smaller and safer runtime image.
- Docker Compose orchestration with service dependencies and database health checks.
- GitHub Actions workflow for cargo check, formatting, linting, and Docker build validation.
- Initial Kubernetes deployment and service files for future scaling and cluster deployment.

3. Need and Relevance of the Project

The project is relevant because DevOps is not only about deployment automation but also about designing applications in a way that supports automation, repeatability, and maintainability. By combining code, infrastructure configuration, container images, and orchestration definitions in one project, the solution shows how software and operations thinking can work together from the start.

It is also a meaningful example for academic evaluation because it turns abstract DevOps concepts into visible deliverables. Students can explain how reverse proxying works, how Docker images are built, how services communicate over a private network, why health checks matter, and how CI pipelines help maintain quality.

Overall Architecture / Workflow

1. High-Level Architecture

According to the submitted high-level design, the application follows a three-tier structure. The user interacts through a browser, which communicates with the Nginx frontend. The frontend acts as both a static file server and a reverse proxy. API requests are forwarded to the Rust backend service. The backend processes the request, performs business logic, interacts with PostgreSQL, and then returns a structured response that flows back through the frontend to the user interface.

This architecture offers strong conceptual clarity. The frontend tier is responsible for presentation and traffic entry, the backend tier for computation and API behavior, and the database tier for persistence. Such separation makes the system easier to understand, test, replace, and scale.

2. Workflow of a Request

- The user opens the application dashboard in the browser through localhost:3000.
- Nginx serves the static UI files and receives any request directed toward API paths.

- Requests beginning with /api/ are forwarded by Nginx to the internal Rust backend container.
- The Axum router matches the endpoint and passes the request to the relevant handler.
- Application logic validates the request and performs the needed database action through SQLx.
- PostgreSQL stores or retrieves the required data and sends the result back to the backend.
- The backend serializes the response into JSON and returns it to the frontend and then to the user.

3. Benefits of the Architecture

The architecture is lightweight yet realistic. It shows how a clean backend can be placed behind a reverse proxy, how a database can be isolated from public exposure, and how the entire system can be reproduced by running a single orchestration command. This offers both educational and practical benefits.

Because the backend is stateless for request processing and the database is persistent through Docker volumes, the design is also well positioned for future scaling. The Kubernetes deployment files included in the repository show that the project already considers a path toward a more distributed environment.

High Level Design Interpretation

The submitted HLD describes the system as a containerized sports platform where the browser communicates first with Nginx, which then forwards API requests to the Rust backend. The backend accesses PostgreSQL for data storage and retrieval. The HLD also emphasizes that the system is designed for efficient communication, improved security, and scalability through private networking and container separation.

Another important point from the HLD is the use of modular responsibilities. The user interface provides access and display, the backend handles request processing and routing, the database stores application data, and Docker Compose manages service dependencies. This makes the HLD not only a diagram of components but also a statement of architectural intent.

The HLD further highlights non-functional goals such as performance, security, and maintainability. These goals are supported by the choice of Rust for high-performance backend processing, Docker for isolated deployment, Nginx for controlled traffic routing, and PostgreSQL for reliable data persistence.

Low Level Design Interpretation

The submitted LLD expands the architecture into component-level responsibilities. It explains the layered backend structure consisting of an API layer, a service layer, a repository layer, and a database layer. This layered approach keeps code organized and makes it easier to test, debug, and extend individual parts of the application.

The LLD also outlines the internal project structure centered around main.rs and supporting modules such as routes, handlers, services, repository, models, and config. Even though the current repository implementation is compact, the LLD provides a sound blueprint for how the project can evolve into a larger maintainable codebase.

The LLD gives special attention to data flow and request processing. It explains that a client request reaches an endpoint, gets validated in the handler, passes through business logic, uses repository access for database communication, and then returns a structured response. This flow is central to understanding how backend systems transform input into reliable output.

Detailed Module and Component Design

1. Frontend Layer

The frontend is packaged as a separate Nginx-based container. Its main responsibilities are to host the static user interface and to act as a reverse proxy for backend traffic. The repository README describes the frontend as a premium standalone interface, while the frontend Dockerfile shows that static assets are copied into the Nginx HTML directory and served over port 80 inside the container.

The custom nginx.conf file defines two main locations. The root location serves the static frontend pages, while the /api/ location forwards requests to the sports-api container. This design avoids direct browser exposure of the backend and resolves cross-origin issues by making the frontend the single visible entry point.

2. Backend Layer

The backend is implemented in Rust and uses Axum for routing, Tokio for asynchronous execution, Serde for JSON serialization and deserialization, and SQLx for PostgreSQL communication. The application defines a Team structure, a CreateTeam input structure, and an AppState structure that holds the database connection pool.

In main.rs, the service reads the DATABASE_URL environment variable, creates a PostgreSQL connection pool, ensures the teams table exists, and then starts an Axum router. The router maps the root path to a status message and maps /teams to both GET and POST operations. This gives the project a functional API foundation while keeping the code easy to understand.

3. Database Layer

PostgreSQL is used as the persistence layer. In the current implementation, the teams table contains an auto-incrementing id, a name field, and a city field. The database is initialized through a table creation query at application startup, which ensures that first-time execution can create the required table automatically.

The database is configured as a separate container using the postgres:15-alpine image. A named Docker volume stores the data outside the lifecycle of the container itself, which means the

information can survive container restarts and rebuilds. This is an important DevOps consideration because it separates runtime services from persistent state.

4. Docker and Orchestration Layer

The backend Dockerfile uses a multi-stage build. In the builder stage, a Rust Bookworm image compiles the application into a release binary. In the runtime stage, a smaller Debian Bookworm Slim image is prepared with only the necessary runtime dependencies, including OpenSSL and libpq support. The binary is copied from the builder image into the runtime image, ownership is assigned to a non-root user, and the service runs with reduced privileges.

The docker-compose.yml file defines three services: frontend, sports-api, and db. The frontend depends on the sports-api service, while the sports-api service depends on the database becoming healthy. The database service includes a pg_isready health check, and all services are connected through a shared Docker network named sports-network.

Tools & Technologies Used

Technology	Role in the Project	Why It Was Used	Observations
Rust	Backend language	High performance, memory safety, and modern systems programming support	Supports reliable API development with strong concurrency primitives
Axum	Web framework	Provides routing and request handling for Rust APIs	Keeps endpoint logic simple and asynchronous
Tokio	Async runtime	Enables non-blocking backend execution	Improves scalability for concurrent requests
Serde	Serialization library	Converts Rust structures to and from JSON	Helps maintain clean request and response contracts
SQLx	Database access	Connects Rust application to PostgreSQL asynchronously	Supports type-safe query execution
PostgreSQL	Relational database	Stores sports data persistently	Reliable ACID-compliant database

			for structured records
Nginx	Frontend server and reverse proxy	Serves UI assets and forwards API calls internally	Improves isolation and simplifies client access
Docker	Containerization platform	Packages application and dependencies into reusable images	Removes environment inconsistency issues
Docker Compose	Service orchestration	Starts and connects multiple services together	Useful for local development and integration testing
GitHub Actions	CI pipeline	Automates checks such as cargo check, fmt, clippy, and Docker build	Improves code quality and repeatability
Kubernetes manifests	Deployment readiness	Provide early cluster deployment configuration	Shows future scaling intent

Repository Structure and Code Understanding

The repository contains multiple top-level folders and files that map directly to the DevOps story of the project. The src folder contains the Rust service code, the frontend folder contains the Nginx-based presentation layer, the k8s folder contains deployment artifacts for future orchestration, and the .github/workflows folder contains continuous integration configuration.

At the backend level, main.rs currently implements routing, database connection management, and data operations for team records in a compact form. While the implementation is intentionally simple, it demonstrates the complete request lifecycle from HTTP entry to database persistence. The project can therefore be presented both as a functioning service and as a foundation for further modular decomposition.

The presence of frontend, Docker, Compose, CI, and Kubernetes artifacts in one repository is important because it shows that the project is not only an isolated API code sample. It is structured as a deployable system that includes runtime, networking, and build automation concerns in the same source base.

API Design and Functional Behavior

1. API Behavior

The backend currently exposes a small but meaningful set of endpoints. The root endpoint returns a health-style message indicating that the sports service is running. The /teams endpoint supports GET for retrieving all team records and POST for creating a new team in the database. Responses are serialized as JSON, which makes the service convenient for frontend integration and easy API testing.

The choice of a compact API surface is suitable for an academic project because it keeps the design focused. The current endpoints are enough to demonstrate request validation, connection pooling, SQL execution, response serialization, and reverse proxy routing without introducing unnecessary complexity.

Endpoint	Method	Purpose	Sample Input	Output
/	GET	Returns a basic service status message	No body	Plain text status
/teams	GET	Fetches all team records from PostgreSQL	No body	JSON array of teams
/teams	POST	Creates a new team with name and city	{"name":"Team A","city":"Indore"}	JSON record of inserted team

2. Sample Processing Logic

When a POST request reaches /teams, the incoming JSON body is deserialized into the CreateTeam structure. The handler uses SQLx to execute an INSERT statement and returns the resulting record as a Team structure. When a GET request reaches /teams, the handler executes a SELECT query and returns all available team rows from the database.

This flow reflects standard RESTful behavior while keeping the code compact and readable. It also allows the frontend to display information in real time after user interaction, which is consistent with the project README description.

Database Design and Data Handling

The current data model is intentionally minimal. The teams table contains three primary pieces of information: id, name, and city. This keeps the schema easy to explain while still showing how structured records are stored in a relational system. PostgreSQL is a strong choice here because it supports constraints, indexing, transactions, and future schema expansion.

The data handling pattern is straightforward. Input JSON is transformed into Rust structures, the backend executes the necessary SQL query, and the resulting row is transformed back into JSON

before being returned to the caller. This pattern demonstrates clean serialization boundaries and supports predictable API behavior.

As the project evolves, the schema can naturally expand to include players, coaches, matches, venues, schedules, scores, and event statistics. The LLD already anticipates such a direction by discussing entities like teams, players, and matches as the broader sports domain model.

DevOps Design Considerations

1. Containerization Strategy

One of the strongest parts of the project is its containerization strategy. The use of Docker ensures that application setup does not depend on a developer manually installing the exact versions of Rust libraries, PostgreSQL, or web server dependencies. Instead, the environment is expressed declaratively through Dockerfiles and Compose configuration.

The multi-stage Docker build further improves the approach by separating build-time requirements from runtime requirements. This reduces the final image size and limits unnecessary tools in the production container, which is a good practice for both efficiency and security.

2. Service Orchestration

Docker Compose acts as the operational glue of the project. It connects containers through a named network, exposes the necessary ports, passes environment variables, mounts a persistent volume for database data, and controls service startup relationships. The backend waits until the database health check passes, which reduces race conditions during startup.

This orchestration layer is especially valuable for local integration testing because it allows the full application to be started using a single command. That makes the project easier to demonstrate, easier to evaluate, and easier to maintain.

3. Continuous Integration Readiness

The GitHub Actions workflow adds another DevOps dimension to the project. The workflow performs cargo check, formatting validation through rustfmt, linting through clippy, and Docker build verification. These steps help identify issues early and reinforce discipline in code quality and build stability.

Even though the workflow is compact, it represents an important move away from purely manual validation. It shows that the project has started adopting an automated quality gate, which is central to DevOps and continuous delivery thinking.

4. Kubernetes Readiness

The repository includes Kubernetes deployment and service manifests for both the sports API and PostgreSQL. The deployment file defines replicas for the API and sets the database URL environment variable. The service file exposes the API through a NodePort and creates a service endpoint for PostgreSQL.

These artifacts indicate that the project already considers a future beyond local Docker Compose execution. While the current scope remains primarily educational and local, the presence of Kubernetes configuration strengthens the argument that the design is extensible and aligned with modern deployment platforms.

Security, Performance, Reliability, and Maintainability

1. Security Aspects

- The runtime image uses a non-root user, reducing unnecessary container privileges.
- The database is not intended to be exposed as a public-facing service in the normal access path; traffic is mediated through internal networking.
- Nginx shields the backend by acting as the public entry point and proxy layer.
- Sensitive runtime values such as database connection strings are managed through environment variables instead of hardcoding inside application logic.

2. Performance Aspects

- Rust offers efficient execution and low runtime overhead.
- Tokio supports asynchronous processing and better concurrency handling.
- Nginx is lightweight and effective for serving static assets and proxying requests.
- PostgreSQL supports performant relational access and can be optimized further with indexing as the schema grows.

3. Reliability Aspects

- Health checks reduce startup timing issues between the backend and the database.
- Persistent volumes protect database data from simple container recreation.
- Separation of containers isolates failures and simplifies troubleshooting.
- Clear environment-based configuration improves repeatability.

4. Maintainability Aspects

The project is maintainable because each major concern is placed in its own layer or artifact. Source code is separate from infrastructure configuration, frontend files are separate from backend logic, and runtime configuration is externalized. This separation improves readability and future teamwork.

The LLD also provides a direction for further modularization into routes, handlers, services, repository, models, and configuration packages. As the project grows, this blueprint can help prevent code from becoming tightly coupled.

Setup, Execution, and Demonstration Flow

- Install Docker Desktop and ensure Docker is running on the host system.
- Clone the repository and open the project folder.

- Run docker-compose up --build to build the images and start all services.
- Wait until the database health check passes and all containers are running.
- Open the application dashboard through http://localhost:3000.
- Use the frontend interface to add teams and view stored data.
- Optionally test the backend directly using curl http://localhost:8080/teams.
- Use docker-compose down to stop the environment gracefully.

This execution flow is simple enough for academic demonstration while still reflecting a realistic multi-container application lifecycle. The command line setup, service logs, and running containers can all be shown during a project presentation to illustrate how the system behaves operationally.

Testing and Validation Approach

1. Validation Strategy

A DevOps project should not only run but also be verifiable at multiple levels. In this project, validation can be observed across code quality, container build quality, service startup reliability, and functional API behavior. The GitHub Actions workflow already provides an automated validation layer by running cargo check, format validation, linting, and Docker build verification whenever changes are pushed or submitted through pull requests.

In addition to CI validation, practical runtime validation is possible through local execution using Docker Compose. When the containers are started, the first checkpoint is whether the PostgreSQL service becomes healthy. The next checkpoint is whether the Rust backend successfully binds to port 8080 and connects to the database. The final checkpoint is whether the frontend becomes accessible and is able to exchange data with the backend through proxied API calls.

2. Types of Testing Relevant to the Project

- Build validation to confirm that the Rust application and Docker images compile successfully.
- Integration validation to confirm that the frontend, backend, and database containers can communicate correctly.
- API validation through browser interactions or direct curl commands to confirm request and response behavior.
- Configuration validation to ensure that environment variables, ports, networks, and volumes are mapped correctly.
- Operational validation to verify restart behavior, health checks, and dependency sequencing.

3. Presentation-Time Demonstration Points

During a project presentation, the team can demonstrate validation in a structured way. First, they can show the repository layout and explain the purpose of each major artifact. Next, they

can run the Compose command, show running containers, and highlight the healthy database state. They can then open the dashboard, create sample team entries, and show the corresponding API response or database-backed display. This makes the demonstration concrete and easy for evaluators to follow.

Deployment Readiness and Operational Perspective

1. Local Deployment Readiness

The current project is strongly prepared for local deployment and academic evaluation. All necessary components for running the system are present in the repository: backend source code, frontend configuration, Docker image definitions, multi-container orchestration, CI workflow, and Kubernetes manifests. This means the project can be cloned and executed without requiring large amounts of manual setup or hidden environment-specific steps.

2. Production-Oriented Next Steps

Although the current implementation is primarily educational, the structure is close to what would be expected in an early production-ready prototype. To move toward production, the team would typically add stronger secrets handling, HTTPS termination, image registry publishing, migration tooling, structured logging, metrics, automated test coverage, and environment-specific deployment profiles. Because the project already separates code from infrastructure and uses containers cleanly, these next steps can be adopted without redesigning the full system.

Results & Output

1. Functional Output

The expected functional result of the project is a working sports platform interface backed by a live Rust API and PostgreSQL database. Users can access the dashboard through the frontend, submit team information, and retrieve saved records from the backend service. The returned information is displayed in real time, demonstrating end-to-end communication across all layers.

At the backend level, successful execution means that the API starts correctly, establishes a database connection, creates the teams table if needed, and serves requests on port 8080. At the platform level, successful execution means that the full system can be built and launched in a consistent way with Docker Compose.

2. Technical Output

- A containerized frontend served through Nginx.
- A Rust backend image built through a multi-stage Dockerfile.
- A PostgreSQL database with persistent storage volume.
- Compose-based orchestration of all services on a shared private network.
- GitHub Actions CI workflow for code and build validation.
- Kubernetes manifest files demonstrating deployment readiness.

3. Key Outcomes

The project successfully demonstrates how DevOps concepts can be applied to a full application rather than to isolated scripts or theory. It shows reproducible builds, operational dependency management, reverse proxy communication, and separation of concerns across tiers.

Another key outcome is educational clarity. Each artifact in the repository supports a clear explanation: Dockerfile for image creation, Compose for orchestration, CI workflow for automated checks, Kubernetes manifests for future deployment, and Rust code for backend service logic.

Challenges, Limitations, and Learning

One challenge in projects of this kind is balancing simplicity and realism. A very large feature set can make the system difficult to explain, while an overly small implementation can appear incomplete. This project manages that balance by focusing on one concrete domain flow while still including the operational artifacts expected in a DevOps-oriented application.

The current implementation remains limited in terms of business features. The API surface is narrow, the data model is small, and enterprise concerns such as authentication, observability, secrets management, migrations, and production-grade deployment policies are not yet fully implemented. These are reasonable limitations for the current academic stage.

At the same time, the project provides strong learning value. It teaches how to build Rust services, connect them to PostgreSQL, configure reverse proxying through Nginx, create multi-stage Docker images, orchestrate services through Compose, and include CI and Kubernetes artifacts in a coherent repository.

Conclusion

The Dockerized Sports Rust Service is a well-structured DevOps project that combines application development and operational design into a single demonstrable system. It uses a three-tier architecture with Nginx, Rust, and PostgreSQL, supported by Docker Compose orchestration, multi-stage image builds, health checks, and CI validation. These elements together show that the project has been designed not only to function but also to be reproducible, maintainable, and ready for future deployment growth.

In summary, the project meets the expectations of a DevOps academic submission by showing how modern tools and practices can be used to package, run, and reason about an application as a system. It demonstrates practical learning in containerization, service orchestration, backend API construction, database connectivity, and deployment thinking.

Future Scope & Enhancements

- Expand the data model to include players, matches, coaches, venues, and schedules.
- Introduce full CRUD support with PUT, PATCH, and DELETE endpoints.

- Add authentication and role-based authorization for admin and viewer access.
- Implement database migrations through a dedicated migration tool rather than startup queries.
- Add observability using logs, metrics, and health dashboards.
- Introduce automated test suites for API behavior and integration scenarios.
- Use secrets management and environment-specific configuration for secure deployment.
- Extend the CI pipeline into a full CD flow with image publishing and deployment stages.
- Deploy the system on a managed Kubernetes environment with ingress and persistent volumes.
- Add richer frontend features such as dashboards, validation messages, search, and sports analytics.

References

1. Docker Official Documentation.
2. Rust Programming Language Documentation.
3. PostgreSQL Documentation.
4. Nginx Documentation.
5. GitHub Actions Documentation.
6. Kubernetes Documentation.